

SYSTEM DESIGN INTERVIEW

Design a URL Shortener

bit.ly-style distributed service for shortening and redirecting URLs

~ 40 minute read

© Study Guide — For personal use only

What's Inside

- 01 · Problem Statement
- 02 · Requirements
- 03 · Capacity Estimation
- 04 · High-Level Architecture
- 05 · Key Generation Strategy
- 06 · Database Design & Schema
- 07 · Write Path Deep Dive
- 08 · Read Path & Redirect
- 09 · Analytics Pipeline
- 10 · URL Expiry & Cleanup
- 11 · Security & Abuse Prevention
- 12 · Key Generation Service (KGS)

- 13** · Scalability & Distribution
- 14** · Failure Modes & Recovery
- 15** · Design Trade-offs
- 16** · Interview Playbook

01

Problem Statement

Requirements and scope

Design a URL shortening service like **bit.ly** or **TinyURL**. The service must convert long URLs into short, memorable links; handle user redirects efficiently; track analytics; and scale to billions of URLs.

Clarifying Questions

Question	Answer
URLs/day?	100 million read-heavy (100:1 read:write ratio)
Expiry?	5 years; lazy expiry on read
Custom aliases?	Yes, optional per-user feature
Scale?	Global; handle single region first
Private URLs?	Yes, with auth + access control
Vanity URLs?	Yes, if available and reserved by user
Analytics required?	Yes: click counts, geographic, referrer

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Shorten	Accept long URL → return short URL (POST /shorten)
Redirect	GET /:key → 301 redirect to original URL
Analytics	Track clicks, referrer, geo, user agent per URL
Expiry	URLs expire after configurable TTL (default 5 years)
Custom Key	Users can specify custom alias if available
Private URLs	Owner-only access with auth token

Non-Functional Requirements

Requirement	Target
Availability	99.9% uptime; graceful degradation
Latency	<100ms for redirect; <200ms for shorten
Consistency	Eventual consistency acceptable; dedup via keys
Storage	~91 TB over 5 years
Throughput	1.2K writes/sec; 115K reads/sec
Scalability	Geo-distributed; $62^7 \approx 3.5$ trillion keys

03

Capacity Estimation

Math for scale

Traffic Estimation

- URLs created/day: **100 million**
- Read:Write ratio: **100:1** (typical for shorteners)
- Write throughput: $100M / 86,400 \text{ sec} = \mathbf{1,157 \text{ writes/sec} \approx 1.2K/sec}$
- Read throughput: $1.2K \times 100 = \mathbf{115,700 \text{ reads/sec} \approx 120K/sec}$

Storage Estimation

- Per URL record: short_key (7B) + original_url (avg 100B) + metadata (100B) $\approx$ 500 bytes
- Daily storage: $100M \times 500B = \mathbf{50 \text{ GB/day}}$
- Retention: 5 years = 1,825 days
- Total: $50 \text{ GB} \times 1,825 = \mathbf{91.25 \text{ TB}}$

Cache Estimation

- Hot URLs: **80% of reads hit ~20% of URLs** (Pareto principle)
- Hot working set is the recently-active URLs over a 24h window — not the 5-year corpus
- 24h reads $\approx 115K/sec \times 86,400 \approx \mathbf{9.9B \text{ requests}}$; 80% (~7.9B) target ~20% of recently-active URLs
- Cardinality of that hot set: ~20–30M unique keys
- Cache size needed: $\sim 25M \text{ URLs} \times 500B \approx \mathbf{12 \text{ GB Redis}}$ (round to ~11.5 GB cluster, TTL = 24h)

KEY

Key Numbers

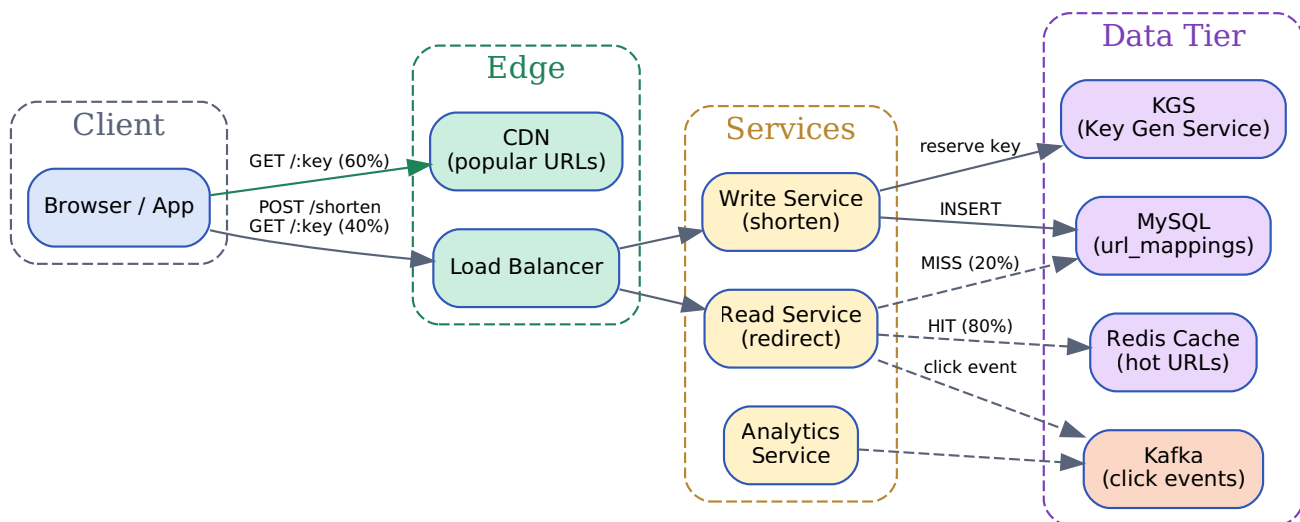
Write throughput: **1.2K/sec** · Read throughput: **120K/sec** · Total storage: **91 TB** · Hot cache: **~12 GB**
Redis cluster

04

High-Level Architecture

System overview

The system is divided into **Edge** (CDN + Load Balancer), **Service Layer** (Write, Read, Analytics), and **Data Tier** (KGS, MySQL, Redis, Kafka). Reads are cached and served from CDN for popular URLs; writes reserve keys from a pre-generated pool.



Request flow: reads served from CDN/Redis where possible; writes reserve a key from KGS, persist to MySQL, and warm the cache.

Architecture Highlights

- **CDN Edge:** caches ~60% of requests (popular URLs); serves 301 redirects
- **Load Balancer:** routes write requests to Write Service, reads to Read Service
- **Write Service:** validates input, reserves short key from KGS, persists to MySQL, warms cache
- **Read Service:** checks Redis (~80% hit rate), falls back to MySQL, logs analytics to Kafka
- **KGS:** maintains pre-generated key pool; assigns keys atomically to avoid collisions
- **MySQL:** persistent storage; sharded by short_key hash for scalability
- **Redis:** in-memory cache for hot URLs; replicated cluster for HA
- **Kafka:** event stream for click logging; async ingestion to Analytics

05

Key Generation Strategy

MD5 vs Base62 vs KGS

Overview

- **Goal:** convert long URL → short, unique, memorable string
- **Candidates:** MD5 hash, Base62 encoding, Pre-generated Key Service (KGS)

Comparison

Approach	Pros	Cons
MD5 hash	Deterministic; no DB lookup	Not memorable; collision possible
Base62 from counter	Compact; sequential	Predictable; needs distributed counter
KGS (pre-generated)	Atomic; no collisions; load-balanced	Requires separate service; pre-warming

Why KGS Wins

- **Atomicity:** keys reserved from a pre-generated pool, no race conditions
- **No collisions:** each key issued exactly once; dedup via DB unique constraint
- **Load balancing:** standby KGS instances for HA; keys pre-loaded on startup
- **Predictability:** latency unaffected by hash computation or counter coordination

Base62 Encoding (Fallback)

```
import string

charset = string.digits + string.ascii_lowercase + string.ascii_uppercase

def to_base62(num):
    if num == 0: return charset[0]
    s = []
    while num > 0:
        s.append(charset[num % 62])
        num //= 62
    return ''.join(reversed(s))
```

```
# 62^7 = 3,521,614,606,208 unique keys
keys = 62 ** 7
```

Key Pool Size Math

- **Alphabet:** 0–9, a–z, A–Z = 62 characters
- **Key length:** 7 characters (bit.ly standard)
- **Total keys:** $62^7 = 3,521,614,606,208$ (3.5 trillion)
- **Years to exhaust:** $3.5T / (100M/day \times 365) \approx 95,910$ years

06

Database Design & Schema

SQL schema and indexing

Primary Table: url_mappings

```
CREATE TABLE url_mappings (  
  short_key      VARCHAR(7) PRIMARY KEY,  
  original_url   TEXT NOT NULL,  
  user_id       BIGINT,  
  created_at     TIMESTAMP DEFAULT NOW(),  
  expires_at     TIMESTAMP,  
  is_custom      BOOLEAN DEFAULT FALSE,  
  click_count    INT DEFAULT 0,  
  last_accessed  TIMESTAMP,  
  UNIQUE KEY uk_user_custom (user_id, short_key) WHERE is_custom,  
  INDEX idx_user_id (user_id),  
  INDEX idx_expires_at (expires_at)  
);
```

Indexing Strategy

- **PRIMARY KEY (short_key)**: fast lookup on redirect; sharded
- **INDEX (user_id)**: for user's URL list queries
- **INDEX (expires_at)**: for background cleanup job
- **UNIQUE (user_id, is_custom)**: prevent duplicate custom URLs per user

Sharding Strategy

- **Shard key**: $\text{hash}(\text{short_key}) \% \text{num_shards}$ (e.g., 256 shards)
- **Reasoning**: distributes writes evenly; enables parallel reads
- **Resharding**: consistent hashing or shard migration tool (Vitess)
- **Shard count**: start with 256; scale to 1024+ as needed

SQL vs NoSQL Decision

Aspect	SQL (MySQL)	NoSQL (DynamoDB / Cassandra)
Schema	Fixed; good for structured data	Flexible; harder to query
Transactions	Full ACID support	Limited (eventual consistency)

Aspect	SQL (MySQL)	NoSQL (DynamoDB / Cassandra)
Joins	Yes; useful for analytics	No; denormalize instead
Scalability	Sharding required; complex	Built-in replication; easier
Cost	Cheaper at scale; self-managed	Expensive at high write throughput

TIP

Recommendation

Use MySQL with sharding: structured data, strong consistency needed, and analytics queries are useful. Shard by short_key hash to distribute load evenly.

07

Write Path Deep Dive

Shortening a URL

Step-by-Step

1. Client submits: `POST /shorten { url, alias?, ttl? }`
2. Validate: URL format, length, Safe Browsing check
3. Custom alias? If provided, check uniqueness in MySQL; reject if taken
4. Reserve key: if no custom alias, request one from KGS (atomic)
5. Persist: `INSERT INTO url_mappings ...`
6. Warm cache: SET Redis key with TTL (24 hours for hot URLs)
7. Async event: publish to Kafka for the analytics pipeline
8. Return: `201 { short_url, created_at, expires_at }`

Optimizations

- **Batch KGS requests:** reserve 100 keys per Write Service instance startup
- **Local cache:** in-memory queue of reserved keys; refill when depleted
- **Async persistence:** write to MySQL; log to Kafka in parallel
- **Dedup check:** UNIQUE constraint backs up application-level checks

Error Handling

- **KGS exhausted:** scale KGS horizontally; widen key length to 8 chars ($62^8 \approx 218T$)
- **MySQL down:** return 503; client retries with backoff
- **Duplicate key:** rare (~ 1 in 3.5T); KGS prevents it; UNIQUE constraint catches
- **Malicious URL:** Safe Browsing returns UNSAFE → reject with 400

08

Read Path & Redirect

Fast redirects

Step-by-Step

1. Browser requests: GET /abc123
2. CDN cache? If HIT (~60% popular URLs), serve 301 directly from edge
3. Miss CDN: route to Read Service
4. Redis lookup: GET abc123 → ~80% hit rate (hot URLs)
5. Cache miss: query MySQL shard for original_url
6. Warm cache: SET Redis with TTL = 24 hours
7. Return: 301 Moved Permanently → original_url
8. Async log: publish click event to Kafka (referrer, geo, user agent)

301 vs 302 Redirect

Type	Use Case	Pros	Cons
301	Permanent move	Browsers cache; reduces load	Cannot retarget; SEO transfers link equity
302	Temporary move	Can retarget any time	No client cache; every hit goes to origin

TIP

Recommendation

Use 301 for bit.ly. Permanent redirects reduce server load by leveraging browser cache. SEO benefit is secondary to performance at this scale.

CDN Caching Strategy

- **Cache key:** GET /:short_key → origin (Read Service)
- **TTL:** 1 hour default; 24 hours for viral URLs
- **Invalidation:** on custom URL update or expiry; manual purge if needed
- **Geographic:** replicate across regions (US, EU, APAC)

09

Analytics Pipeline

Click tracking and insights

Architecture

- Click events logged asynchronously by Read Service to Kafka
- Kafka topic: `url_clicks` (partition by `short_key` for ordering)
- Stream processor: Apache Flink or Kafka Streams
- Aggregations: count, geo, referrer, user agent; time windows (1min / 1h / 1d)
- Storage: Cassandra (time-series) for analytics; Redis for dashboard hot reads

Event Schema

```
{
  "short_key": "abc123",
  "user_id": 12345,
  "timestamp": "2026-03-18T10:30:45Z",
  "ip_address": "203.0.113.42",
  "country": "US",
  "referrer": "twitter.com",
  "user_agent": "Mozilla/5.0 ...",
  "click_id": "<uuid>"
}
```

Metrics

- **Total clicks:** sum by `short_key` over time window
- **Unique visitors:** HyperLogLog for approx distinct count
- **Geographic heatmap:** clicks by country; top 10 regions
- **Referrer breakdown:** top referrers by click count
- **Trending:** clicks in last 1 hour; identify viral URLs

INSIGHT

HyperLogLog

For unique-visitor counts use HyperLogLog (a probabilistic data structure). Uses ~1.5 KB per key, handles billions of unique values with ~2% error. Trade accuracy for memory efficiency.

10

URL Expiry & Cleanup

Managing data retention

Expiry Mechanism

- **Default TTL:** 5 years from creation
- **Lazy expiry:** on read, check `expires_at`; if expired, return 404
- **No explicit delete:** reduces write load; cleanup runs async
- **Custom TTL:** users can specify shorter expiry (e.g., 7 days)

Background Cleanup Job

- **Frequency:** daily at 2 AM UTC (off-peak)
- **Cursor pagination:** scan shards incrementally to prevent long locks
- **Delete:** `DELETE FROM url_mappings WHERE short_key IN (...)`
- **Evict cache:** DEL from Redis for deleted keys
- **Log:** track deleted keys for analytics retention

```
-- Background cleanup (runs daily, off-peak)
SELECT short_key, original_url FROM url_mappings
WHERE expires_at < NOW()
LIMIT 100000
FOR UPDATE SKIP LOCKED;

-- Cursor-based pagination per shard
SELECT short_key FROM url_mappings
WHERE shard_id = ? AND short_key > ?
ORDER BY short_key LIMIT 10000;
```

Cold Archive

- **Retention:** deleted URLs in S3 Glacier for 7 years
- **Cost:** ~\$0.004/GB-mo Glacier vs ~\$0.023/GB-mo S3 Standard vs ~\$0.10+/GB-mo MySQL
- **Query:** archive by month; allow user export for compliance

Compliance

- **GDPR right-to-delete:** anonymise `user_id`, retain `short_key` for reference
- **Audit log:** log all deletions (`short_key`, `user_id`, `deleted_at`, reason)
- **Retention:** keep audit logs for 2 years minimum

11

Security & Abuse Prevention

Protecting the system

Malicious URL Detection

- **Google Safe Browsing API:** phishing, malware, unwanted software
- **Call on shorten:** synchronously block unsafe URLs
- **Fallback:** log to Kafka for manual review if API unavailable
- **Rate limit:** 5 req/sec per IP for Safe Browsing calls

Rate Limiting

- **Per-user:** token bucket; 1,000 shortened URLs/day per user
- **Per-IP:** 100 req/min for anonymous users
- **Implementation:** Redis INCR + EXPIRE; check on every request
- **Tiers:** premium 10K/day, enterprise custom limits

```
# Token bucket (Redis)
key = f'rate_limit:{user_id}'
current = redis.get(key) or 0
if current >= limit:
    return 429 # Too Many Requests
redis.incr(key)
redis.expire(key, 86400) # 1 day
```

Private URLs & Access Control

- **Private flag:** boolean column in url_mappings
- **Access check:** on read, if private, verify auth token matches owner
- **Shared access:** access_control table (url_id, user_id, expires_at)
- **Token:** JWT with user_id, url_id, expiry; signed

DDoS & SQL-Injection Protection

- **CDN DDoS:** Cloudflare or AWS Shield; absorbs volumetric attacks
- **App-layer:** rate limit per IP; detect patterns (same short_key)
- **Auto-scale:** Read Service scales on QPS (threshold 200K/sec)
- **SQL safety:** parameterized queries (sqlalchemy/PDO); input validation `^[0-9a-zA-Z]{1,7}$`

12

Key Generation Service (KGS)

Generating unique keys

Architecture

- **KGS instance:** stateless service with in-memory key pool
- **Key pool DB:** separate MySQL instance for reserved counter ranges
- **Pre-generation:** KGS generates 100K keys at startup
- **Local cache:** in-memory queue; refill when < 10K remaining
- **Standby KGS:** secondary instance for failover (heartbeat every 5 sec)

Key Pool Schema

```
CREATE TABLE key_pool (  
  pool_id          INT PRIMARY KEY AUTO_INCREMENT,  
  start_counter    BIGINT NOT NULL,  
  end_counter      BIGINT NOT NULL,  
  status           ENUM('AVAILABLE', 'RESERVED', 'EXHAUSTED'),  
  reserved_by      VARCHAR(50),  
  reserved_at      TIMESTAMP,  
  INDEX idx_status (status)  
);
```

Reservation Flow

1. KGS startup: `SELECT FROM key_pool WHERE status='AVAILABLE' LIMIT 1`
2. Reserve range: `UPDATE key_pool SET status='RESERVED', reserved_by=? WHERE pool_id=?`
3. Fetch keys: load `start_counter` to `end_counter`; convert to Base62
4. Local cache: store in-memory queue; increment per request
5. Refill: when queue < 10K keys, `SELECT` next available pool

Edge Case: KGS Exhaustion

- **Problem:** all 62^7 keys exhausted (unlikely in > 90,000 years at this rate)
- **Solution 1:** increase key length to 8 ($62^8 \approx 218$ trillion)
- **Solution 2:** recycle expired keys (rescan and return to pool)
- **Monitoring:** alert when pool utilization > 90%

13

Scalability & Distribution

Growing to billions

Horizontal Scaling

- **Write Service:** stateless; add instances behind load balancer
- **Read Service:** stateless; scale independently from writes
- **KGS:** multiple instances; each owns a non-overlapping pool range
- **Database:** shard by short_key hash (256–1024 shards)

Database Sharding

- **Shard key:** $\text{hash}(\text{short_key}) \% 256$
- **Replication:** primary + 2 replicas per shard (3-way)
- **Read replicas:** serve analytics queries; eventual consistency OK
- **Write failover:** automatic promotion (Orchestrator)

Global Distribution

- **Primary region:** US-East (writes; strong consistency)
- **Read replicas:** US-West, EU, APAC (1–2 sec replication lag)
- **CDN:** global edge caches popular URLs
- **Kafka:** multi-region cluster; replicate analytics across regions

Cache Strategy

- **Redis cluster:** 16 nodes, sharded by short_key (~12 GB hot working set)
- **Cache-aside:** Read Service checks Redis; miss = MySQL query + cache fill
- **TTL:** 24 hours for hot URLs; 1 hour for cooler ones
- **Invalidation:** manual purge on custom URL update; periodic refresh

INSIGHT

Consistent Hashing

Use consistent hashing for cache and database sharding. Allows adding/removing nodes with minimal key redistribution (only 1/N keys move). Examples: Jump Hash, Ketama.

14

Failure Modes & Recovery

What can go wrong

Failure	Impact	Detection	Mitigation
Redis down	Cache miss; ~5x MySQL load	Redis health check	Failover to replica; degrade gracefully
MySQL shard down	Reads/writes to that shard fail	Connection timeout	Auto-promote replica; reroute hot keys
KGS exhausted	Cannot create new URLs	KGS error metric	Scale KGS; increase key length; recycle
CDN edge down	Slow redirects from that region	CDN provider alert	Reroute to nearest edge; fall back to origin
Kafka down	Analytics not logged; writes still work	Broker dead alert	Async retry; local buffer + replay
Safe Browsing timeout	Shorten request slow	HTTP timeout (>2s)	Allow with 'pending review' flag; async re-check
Network partition	Cross-region unavailable	Latency spike; conn errors	Local DC serves; eventual consistency on repair

Disaster Recovery

- **RTO:** <30 minutes for any single service
- **RPO:** <1 minute for analytics (Kafka); <5 min for URLs (MySQL)
- **Backups:** incremental MySQL hourly; S3 30-day retention
- **Failover:** automatic for cache and KGS; manual for DB to ensure consistency

WARNING

Graceful Degradation

In failures, prioritize redirects (the core function). Degrade analytics logging and custom aliases first. Always return 200 for known URLs, 404 for unknown; never 5xx unless truly critical.

15

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Redirect type	301 Permanent	Browser cache reduces load; cannot retarget. 302 allows flexibility but increases origin traffic.
Cache consistency	Eventual	Stale data briefly; faster reads. Strong consistency would slow the redirect path.
Analytics	Async (Kafka)	Delayed by 1–2 sec; doesn't block shorten. Sync logging would add ~50 ms to every shorten.
Database	MySQL sharded	Operational complexity; strong consistency. NoSQL is easier to scale but weaker on consistency.
Key generation	Pre-generated (KGS)	Extra service; no collisions. Base62 counter is simpler but needs distributed coordination.
Expiry	Lazy	Stale rows briefly; very low overhead. Active scan adds DB pressure.
CDN coverage	60% popular URLs	Reduced origin load; ~40% miss rate. 100% coverage costs more and adds invalidation pain.
Sharding	256 shards	Good parallelism; manageable ops. 64 contention; 1024+ overhead.

This guide distills a 40-minute interview into a structured narrative. Use it to practice explaining system design clearly, defending trade-offs, and handling follow-ups with confidence.

45-Minute Interview Outline

1. **Intro (2 min):** clarify requirements: 100M URLs/day, 100:1 read:write, 5-yr retention, global
2. **Capacity (5 min):** 1.2K writes/sec, 120K reads/sec, 91 TB storage, ~12 GB cache
3. **High-level arch (3 min):** CDN, LB, services, data tier
4. **Write path (8 min):** KGS, MySQL, Redis warming; explain atomicity
5. **Read path (7 min):** cache hit-rate, redirect type, analytics logging
6. **Deep dive (12 min):** KGS counter ranges & HA, sharding, or analytics
7. **Scalability (5 min):** horizontal scaling, global distribution, monitoring
8. **Failures (3 min):** top 3 failure modes; mitigation for each

Key Talking Points

- “1.2K writes/sec, 120K reads/sec” — demonstrates capacity math
- “ $62^7 = 3.5$ trillion keys” — why KGS works and won't exhaust
- “91 TB storage” — justifies database sharding strategy
- “KGS pre-generates keys” — prevents collisions; enables atomicity
- “301 + CDN cache” — reduces load; handles 100:1 ratio
- “Async Kafka for analytics” — doesn't block the critical path
- “Eventual consistency acceptable” — explicit trade-off rationale

Common Follow-ups & Answers

- **Q: How do you prevent collisions?** A: KGS reserves keys from a pre-generated pool; every key issued once. UNIQUE constraint catches any duplicates.
- **Q: What if MySQL goes down?** A: Reads to that shard fail; auto-failover (Orchestrator) promotes a replica. Writes buffer locally and retry on recovery.
- **Q: How do you scale the cache?** A: Redis cluster, sharded by `short_key` with consistent hashing; adding nodes moves only 1/N of keys.
- **Q: 301 vs 302 cost?** A: 301 reduces origin traffic by ~80% via browser cache. 302 reaches the origin every time. At 120K reads/sec, 301 wins by a wide margin.

- **Q: Malicious URLs?** A: Block at shorten via Safe Browsing; rate-limit per user (1000/day) to prevent abuse.

KEY

Memorize These Numbers

1.2K writes/sec · 120K reads/sec · $62^7 = 3.5\text{T}$ keys · 91 TB over 5 years · 80% Redis hit rate · 60% CDN hit rate · ~12 GB hot cache.